



CLARUSWAY

WAY TO REINVENT YOURSELF



clarusway



clarusway



clarusway



clarusway



clarusway

1

Inheritance

Lesson:
JAVA Chapter 24

Inheritance (Kalıtım/Miras)

Kalıtım (Inheritance), bir Obj/Class'ın **fields** ve **method'larını** bir başka Obj/Class'a aynen aktarması ya da bazı özellik ve işlevlerini diğer sınıfların kullanmasına izin vermesi şeklinde tanımlanır.

Üst Sınıf (Süper Class) özelliklerini alt sınıflarda (**sub Class**) kullanmak için **extends** edilmelidir.

Extends: Herhangi bir Class'ın fields ve method'larını miras olarak alması için kullanılan keyword'dur.

Kalıtım (Inheritance) Faydaları

- Önceden oluşturduğumuz Class'lar, projelerimiz üzerinde denenmiş ve hatasızdır. Bu yüzden kalıtım ile oluşturduğumuz bir alt Class'ı kullanacağımız zaman hata ile karşılaşma ihtimalimiz azalır.
- Daha önceden yazılmış kod parçacıklarını tekrar tekrar yazma sıkıntısından kurtararak alt Class'larda kullanmamıza imkan sağlar. Bu durumda zaman ve emek tasarrufu sağlanır.
- Gerçekleştireceğimiz bir değişikliği her sınıfta ayrı ayrı uğraşmak yerine **super class** üzerinde güncelleme yapma esnekliği oluşturur.
- **if, else, switch** gibi kontrol ifadelerinin kullanıldığı Class'lar kötü tasarım örnekleri olarak ele alındıklarından, kalıtımın nimetlerinden faydalanabiliriz.

Inheritance ile **child class**, **parent class**'daki public veya protected primitive dataları, objectleri, veya metodları kullanabilir.



Inheritance (Kalıtım/Miras)

Inheritance ile code'un tekrar tekrar kullanılabilmesi (**reusability**) sağlanır.
Geneli kapsayan class üyeleri parent class'da, spesifik olanlar ise child class'larda create edilir.

* TRICK :)

- * Child classlar **public** ve **protected** data'ları inherit edebilir.
- * **Private** data'lar inherit edilemez.
- * **Default** data'lar child ve parent aynı package'da ise inherit edilebilirler.
- * **Static Methods** veya variable'lar inherit edilemezler.
- * Üst sınıfın birden fazla alt sınıfı bulunabilir. Fakat bir alt (sub) sınıfın birden fazla üst (super) sınıfı bulunamaz.
- * Kalıtımda, **sub class** özellikleri **süper class'dan** alır. Ancak **süper class** , özellikleri hiçbir zaman **sub class'dan** almaz.



Inheritance (Kalıtım/Miras)

Superclass ve Subclass

- **Super Class (Üst Sınıf):** İçerisinde bulunan özellikleri kalıtım yolu ile miras alınan temel Class'tır.
- **Sub Class (Alt Sınıf):** Üst Class türetilmiş bir Class'tır. Bağlı olunan üst Class içerisinden bir veriyi işlemek için çekmek istediğimiz zaman **super** tanımını kullanır.

```
SuperClassTest.java
1 public class SuperClassTest {
2     public int intValue;
3     public void testMethod() {
4     }
5
6     class SubClassTest extends SuperClassTest {
7         public void subMethod() {
8             super.testMethod();
9             super.intValue;
10        }
11    }
12 }
```



Inheritance (Kalıtım/Miras)



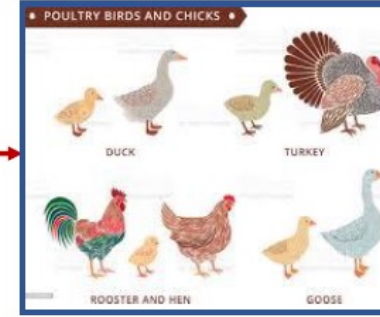
Hayvanlar



Baliklar



Kuslar



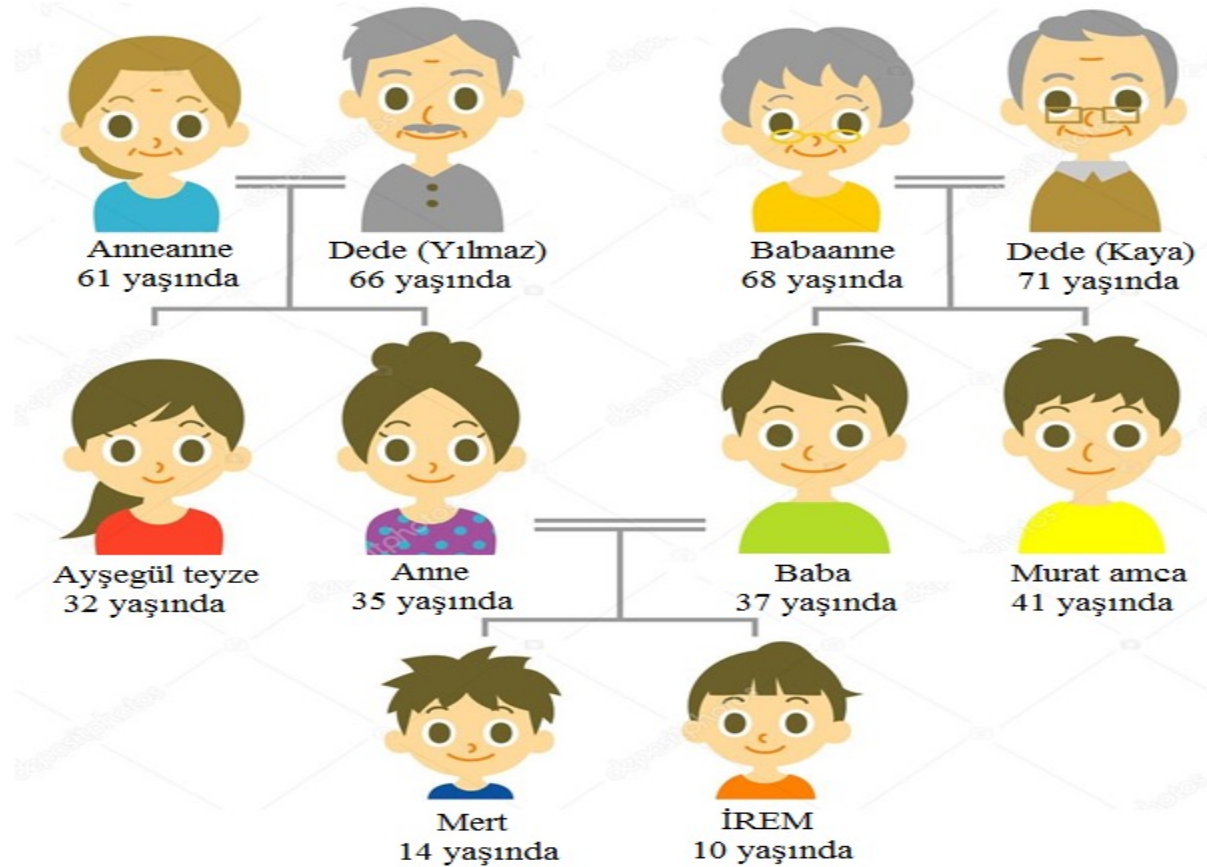
Kumes Hayvanlari



Avcı Kuslar



Inheritance (Kalıtım/Miras)



Inheritance (Kalıtım/Miras)

Interview Question

Interview

Niçin Inheritance kullanırız?

Inheritance sayesinde parent olarak tanımlanan class (ve onun parent class'larındaki) **protected/public** class üyelerini kullanabiliriz. (reusability).

Inheritance'in faydaları nelerdir?

- 1: Tekrarlardan kurtuluruz.
- 2: Daha az kod yazarak işlemlerimizi yapabiliriz.
- 3: Kolayca update yapabiliriz.
- 4: Application'ın bakımı ve sürdürülmesi (maintenance) kolaylaşır.



Inheritance

```
public class Company {  
  
    public static int sayac = 100;  
    public int id;  
    public String isim;  
    public String soyisim;  
    public String adres;  
    public String tel;  
  
    public int idAta() {  
        this.id = sayac;  
        sayac++;  
        return id;  
    }  
}
```

```
public class It extends Company {  
    String kidem;  
    int saatUcreti;  
    int maas;  
  
    public int maasHesapla() {  
        int maas = saatUcreti * 111 * 30;  
        return maas;  
    }  
}
```

```
public class Developer extends It {  
  
    public static void main(String[] args) {  
        Developer backend = new Developer();  
        backend.isim = "Hardy";  
        backend.soyisim = "Miller";  
        backend.tel = "3579113";  
        backend.saatUcreti = 99;  
        backend.maas = backend.maasHesapla();  
        backend.id = backend.idAta();  
  
        Developer frontend = new Developer();  
        frontend.isim = "Ashley";  
        frontend.soyisim = "Miller";  
        frontend.tel = "987654";  
        frontend.saatUcreti = 93;  
        frontend.maas = frontend.maasHesapla();  
        frontend.id = frontend.idAta();  
  
        System.out.println(backend.id + " " + backend.maas);  
        System.out.println(frontend.id + " " + frontend.maas);  
    }  
}
```

```
public class QaTester extends It {  
  
    public static void main(String[] args) {  
        QaTester qa1 = new QaTester();  
        qa1.isim = "Elly";  
        qa1.soyisim = "White";  
        qa1.tel = "888776654";  
        qa1.saatUcreti = 70;  
        qa1.maas = qa1.maasHesapla();  
        qa1.id = qa1.idAta();  
  
        QaTester qa2 = new QaTester();  
        qa2.isim = "Garryy";  
        qa2.soyisim = "Black";  
        qa2.tel = "987654";  
        qa2.saatUcreti = 77;  
        qa2.maas = qa2.maasHesapla();  
        qa2.id = qa2.idAta();  
  
        System.out.println(qa1.id + " " + qa1.maas);  
        System.out.println(qa2.id + " " + qa2.maas);  
    }  
}
```



Inheritance Type

Beş tür **inheritance** vardır.

- Simple **inheritance** (veya) Single **inheritance**
- Multiple **inheritance**
- Multi Level **inheritance**
- Hierarchical **inheritance**
- Hybrid **inheritance**

Java programlama dili, Multiple **inheritance** türünü desteklemez. Single **inheritance** destekler. Ancak, **interface** kavramı ile bir alternatif sağlar. Bir child class'ın sadece bir tane parent class'ı olabilir.

Bir çocuğun ailesi bir tane olur.



Inheritance Türleri

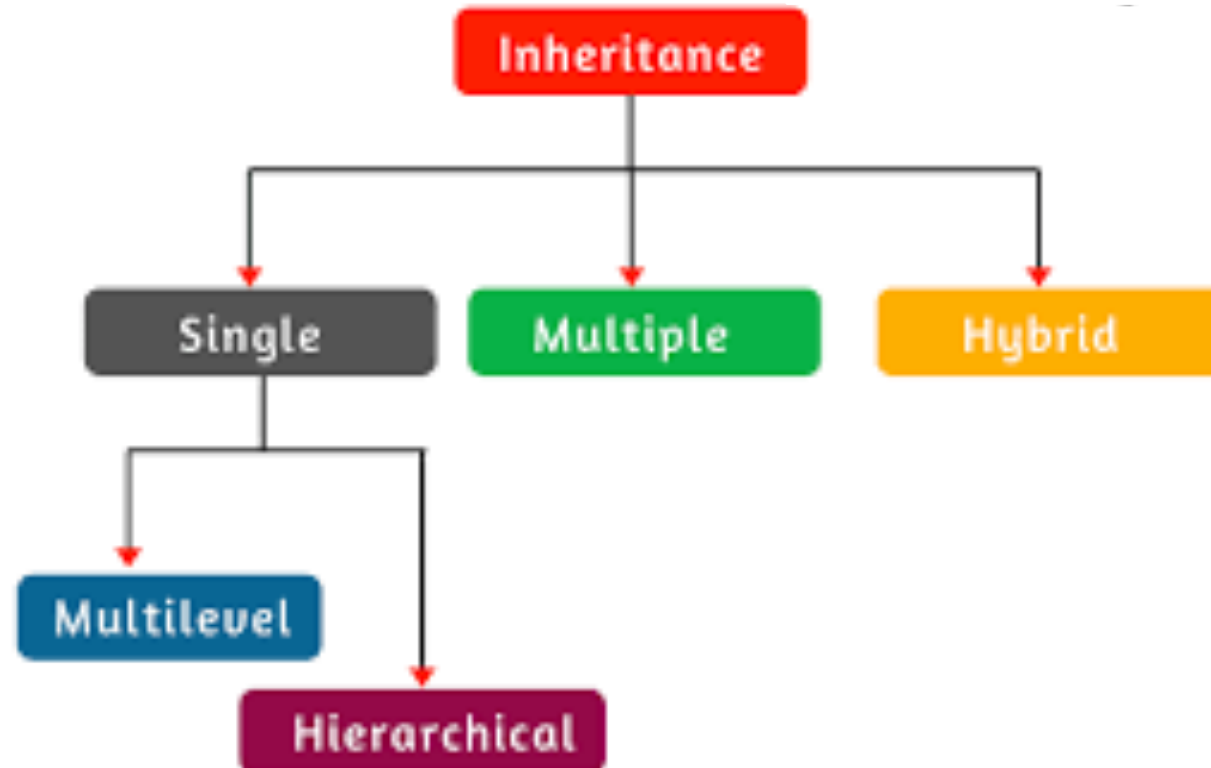
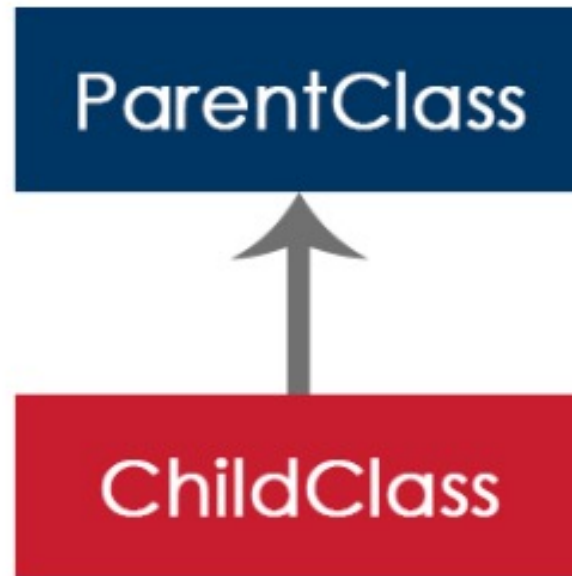


Fig: Classification of Inheritance in Java



Inheritance Türleri

Simple Inheritance



Bir çocuğun bir tane ailesi olur.

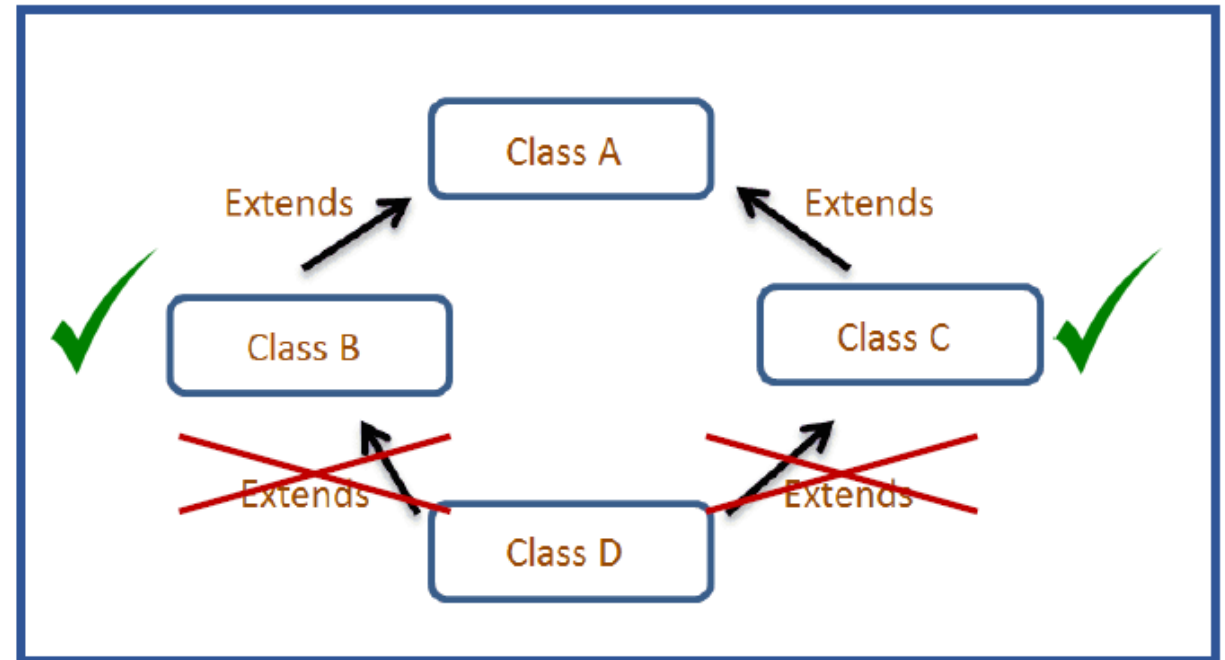
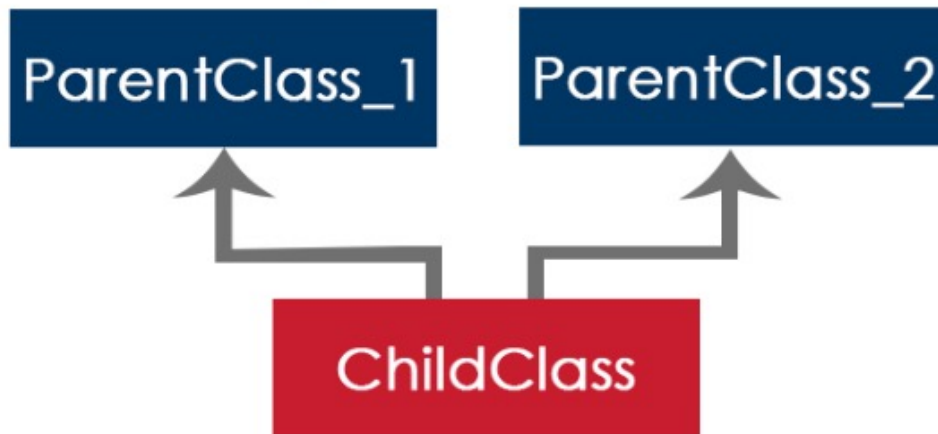


Inheritance Türleri

Multiple Inheritance

Java programlama dili, Multiple **inheritance** türünü desteklemez.

Multiple Inheritance



Bir çocuğun birden çok ailesi olamaz.



Inheritance Türleri

Multi Level Inheritance

JAVA Inheritance zincirini destekler. Bir child class'ın sadece bir tane parent class'ı olabilir. Aynı zamanda parent class zinciri.

Multi Level Inheritance



İnsanlardaki soy ağacı gibi, child class'ın parent'i ve grand parent'leri olabilir.

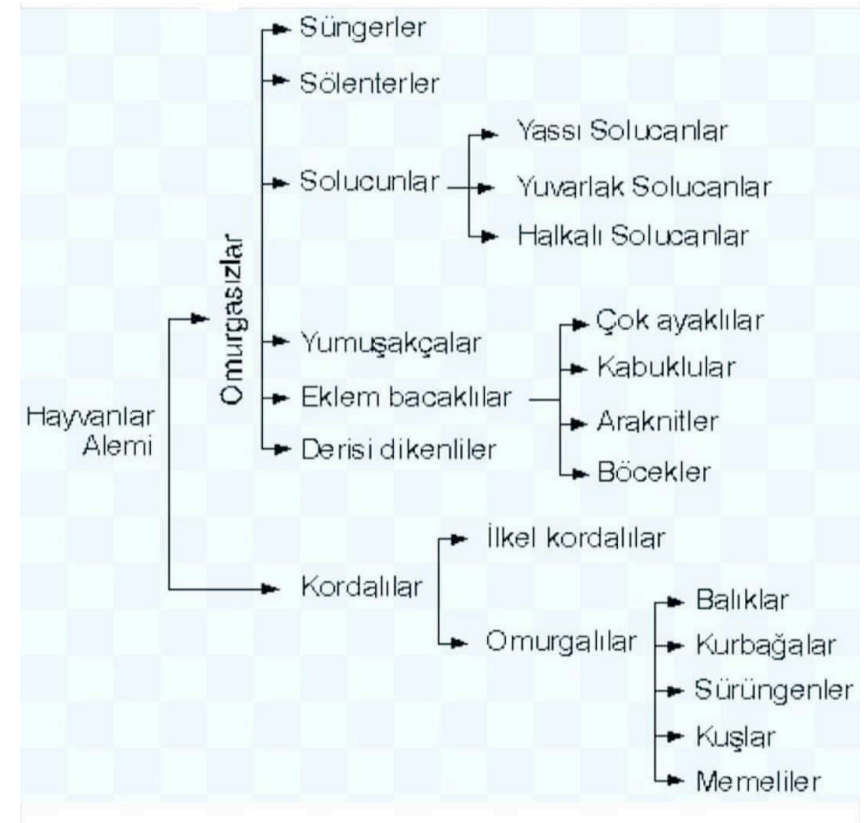
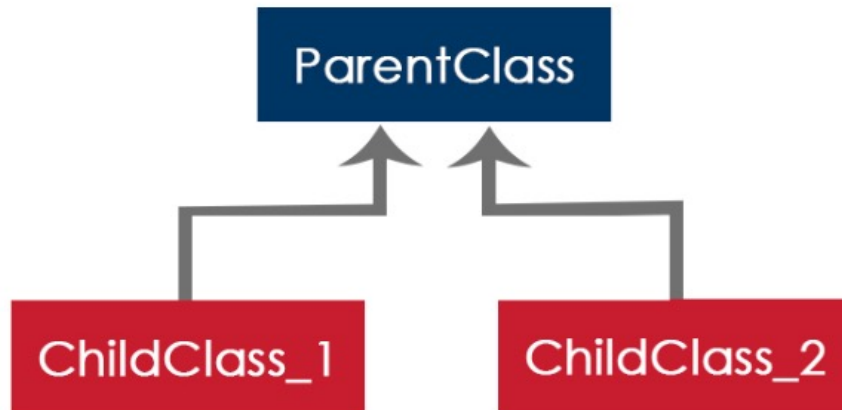


Inheritance Türleri

Hierarchical Inheritance

Bir parent class'ın birden fazla child class'ı olabilir.

Hierarchical Inheritance



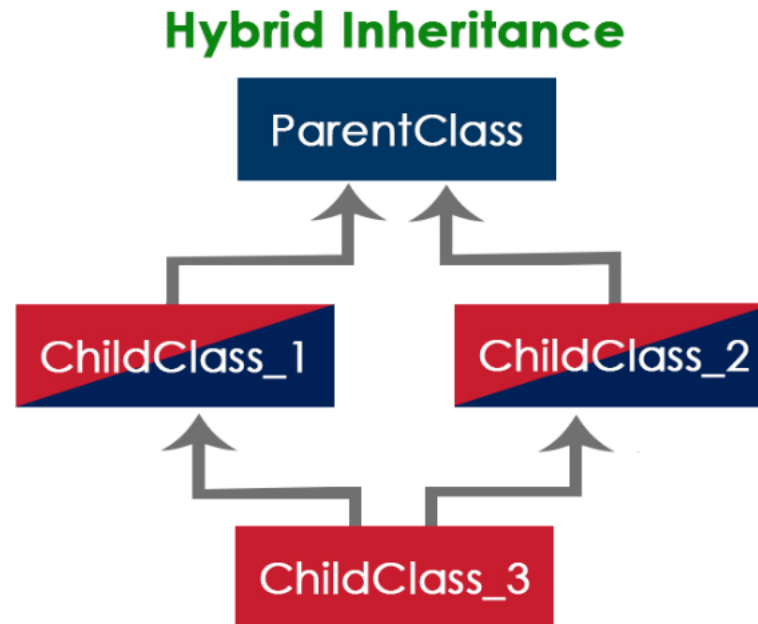
Inheritance Türleri

Hybrid Inheritance

Bir class bir parent'ın child'ı olabildiği gibi aynı zamanda başka bir child class'ın da parent'ı olabilir.

Hybrid Inheritance, birden fazla kalıtım türünün birleşimidir.

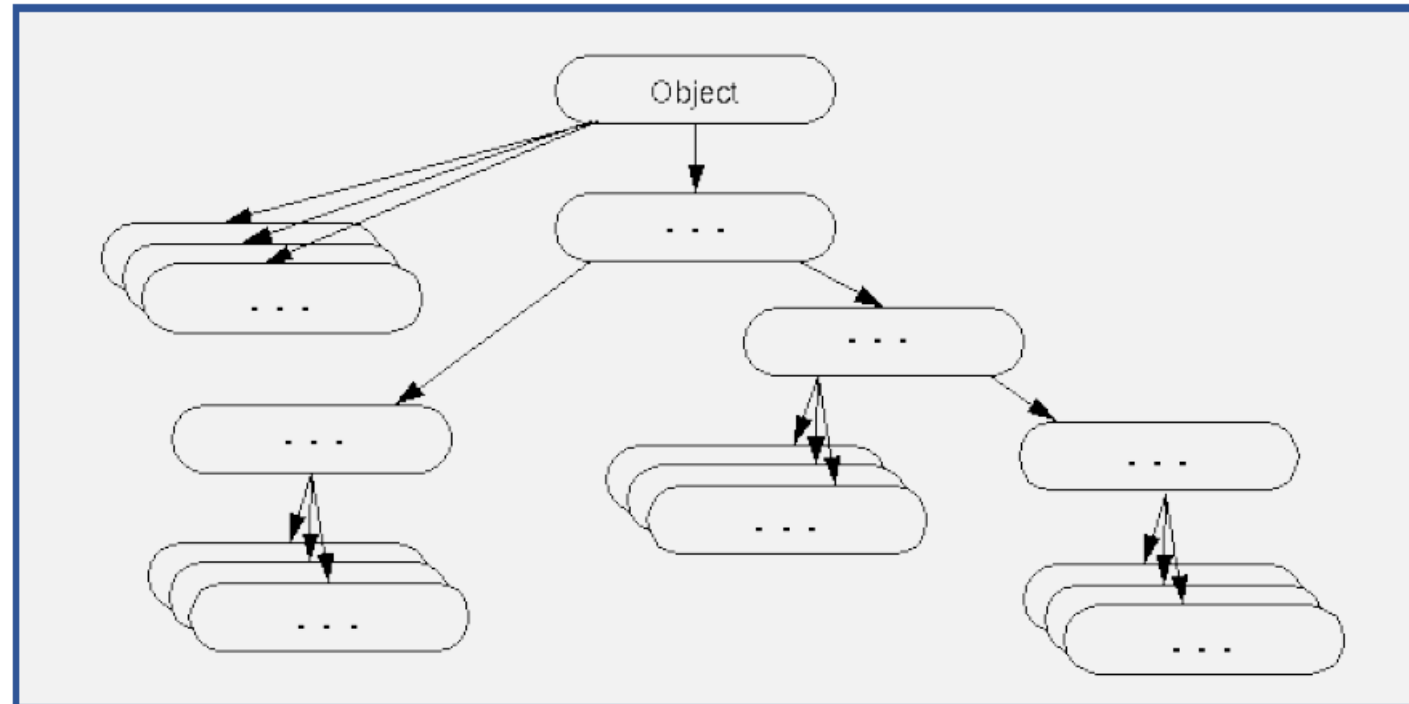
Herhangi bir kombinasyonu birden çok kalıtımla tekli, çoklu kalıtımla çok seviyeli vb. kullanabiliriz.



Inheritance Türleri

JAVA'nın tüm class'ları **Object Class**'dan (Hz. Adem) inherit edilir.

Object Class bütün class'ların parent'idir ve **Object Class** parent'i olmayan tek class'dır.



IS-A () & HAS-A () Relationship

IS-A ilişkisi anlatıldığı gibi aslında kalıtım ilişkisidir. Süper Class'a bağlanan sub class'lar arasında IS-A ilişkisi vardır.

Müdür is-a çalışan
Öğretmen is-a çalışan
Memur is-a çalışan

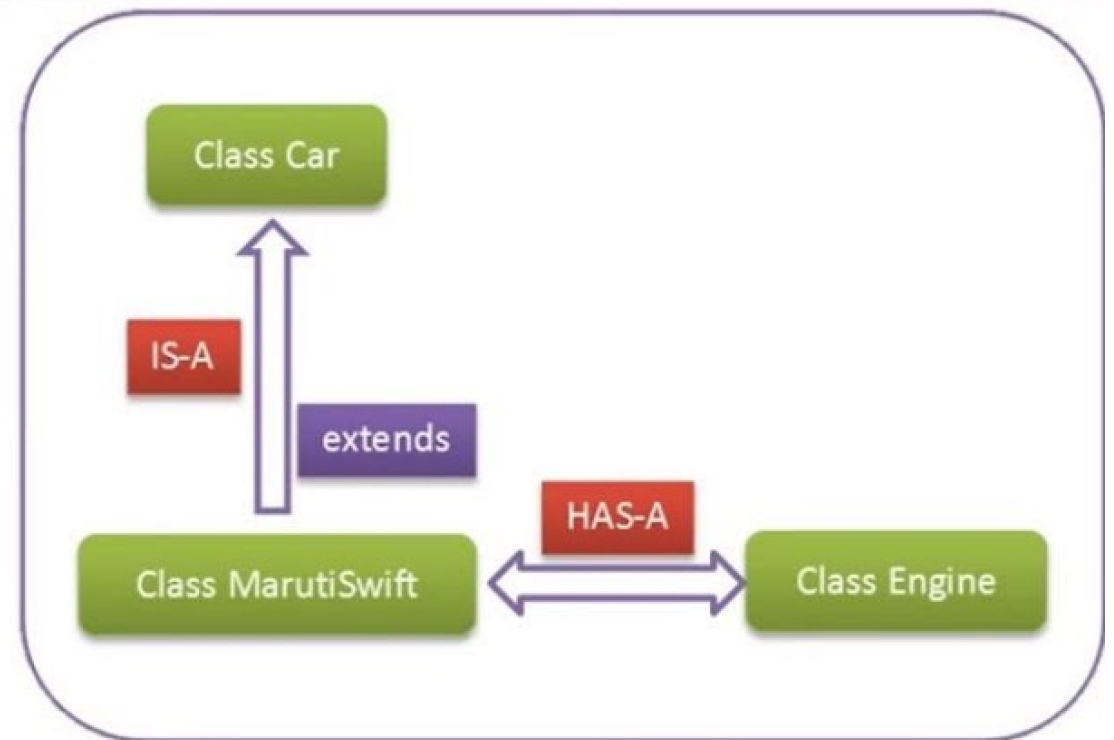
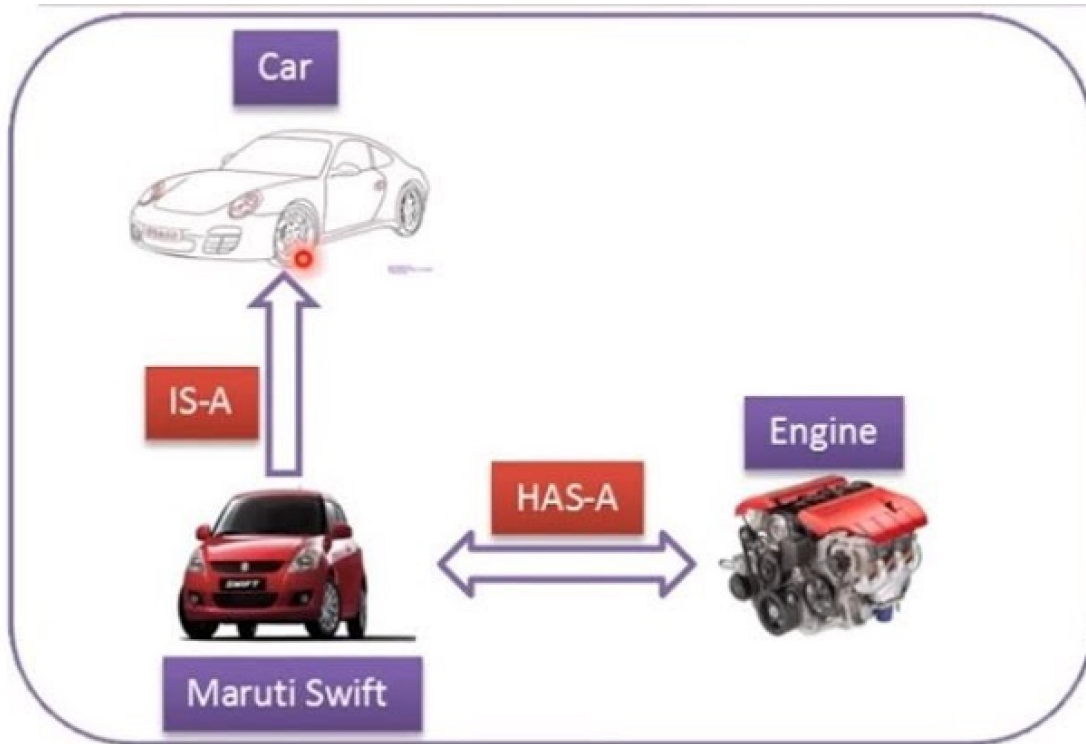
HAS-A ilişkisi ise bir class içerisinde başka bir class'dan obje oluşturulup o objenin özelliklerinin kullanılması demektir. Objenin oluşturulduğu class diğer class'a ait bir objeye sahiptir dolayısıyla aralarında sahip olmakla alakalı bir ilişki vardır, zaten HAS, HAVE (sahip olmak) fiilinin geniş zamanda çekilmiş halidir.

HAS-A ilişkisi JAVA'da kodun yeniden kullanılabilirliği için kullanılır.

Bir extends keyword görülen yerde, class **IS-A** ilişkisine sahiptir.



IS-A () & HAS-A () Relationship



Inheritance'da Constructor Call

1) Bir class'da constructor çalıştığında her constructor'ın ilk satırındaki default `super()` keyword ile parent class'daki constructor call edilir.

```
public class MotorluArac extends Arac {  
    MotorluArac() {  
        System.out.println("MotorluArac constructor çalıştı :) ");  
    }  
}
```

extends

```
public class Arac {  
    Arac() {  
        System.out.println("Arac constructor çalıştı :) ");  
    }  
}
```

extends

```
public class Otobus extends MotorluArac {  
    Otobus() {  
        System.out.println("Otobus constructor çalıştı :) ");  
    }  
    public static void main(String[] args) {  
        Otobus setra = new Otobus();  
    }  
}
```

output

```
Arac constructor çalıştı :)  
MotorluArac constructor çalıştı :)  
Otobus constructor çalıştı :)
```



Inheritance'da Constructor Call

```
public class Otobus extends MotorluArac {  
  
}
```

*** TRICK :)**

Aşağıdaki tüm classlar aynıdır.

```
public class Otobus extends MotorluArac {  
    public Otobus() {  
    }  
}
```

```
public class Otobus extends MotorluArac {  
    public Otobus() {  
        super();  
    }  
}
```



Inheritance'da Constructor Call

2) parent (super) class'da super() ile call edilen constructor yoksa JAVA CTE verir.

```
public class MotorluArac extends Arac {  
  
    MotorluArac() {  
        System.out.println("MotorluArac constructor çalıştı :) ");  
    }  
}  
  
public class Otobus extends MotorluArac {  
  
    Otobus() {  
        super(17);  
        System.out.println("Otobus constructor çalıştı :) ");  
    }  
  
    public static void main(String[] args) {  
        Otobus setra = new Otobus();  
    }  
}
```

extends
↑



Inheritance'da Constructor Call

Output Nedir?

```
public class QaTester extends Clarusway {

    QaTester(int offer) {
        super();
        System.out.println("QaTester child class parametreli constructor çalıştı :) ");
    }

    QaTester() {
        this( offer: 99);
        System.out.println("QaTester child class parametresiz constructor çalıştı :) ");
    }

    public static void main(String[] args) {
        QaTester ashley = new QaTester();
    }
}
```

```
public class Clarusway {

    Clarusway() {
        System.out.println("Clarusway Parent class constructor çalıştı :) ");
    }
}
```

```
Clarusway Parent class constructor çalıştı :)
QaTester child class parametreli constructor çalıştı :)
QaTester child class parametresiz constructor çalıştı :)
```



Inheritance'da Constructor Call

Aşağıdaki code blok'daki CTE'ler nasıl handle edilir ve output ne olur?

```
class Clarusway {  
    Clarusway(int offer) {  
        System.out.println("Clarusway class "+offer);  
    }  
}  
  
public class QaTester extends Clarusway{  
    QaTester(int offer){  
        System.out.println("QaTester class "+offer);  
    }  
  
    public static void main(String[] args) {  
        QaTester ashley=new QaTester();  
    }  
}
```



Inheritance'da Constructor Call

```
class Clarusway {  
    Clarusway(int offer) {  
        System.out.println("Clarusway class "+offer);  
    }  
}  
class QaTester extends Clarusway{  
    QaTester(int offer){  
        super(offer: 99);  
        System.out.println("QaTester class "+offer);  
    }  
  
    public static void main(String[] args) {  
        QaTester ashley=new QaTester(offer: 101);  
    }  
}
```

→
output

```
Clarusway class 99  
QaTester class 101
```



Inheritance'da Class Üyelerini Çağırma

super keyword **super (parent) class'dan** **this** keyword **sub(child) class'dan** variable call etme için kullanılır.

this keyword parent class'dan da variable call edebilir ancak her iki class'da da aynı isimli variable olabileceği için tavsiye edilmez. **this** ve **super** keyword'leri ilk satırda yazılma şartı yoktur dolayısıyla birlikte kullanılabilir.

super() ve **this()** keyword'leri constructor call etmek için kullanılır ve constructor'un ilk satırına yazılmalıdır. Dolayısıyla birlikte kullanılamazlar.



Inheritance'da Class Üyelerini Çağırma

```

Clarusway class consturtor çalıştı :)
QaTester child class constructor çalıştı :)
99
101
1
1
3
Ashley
Jack
sales force
sales force
Miller
  
```

```

public class QaTester extends Clarusway {
    int offer=99;
    int batch = 3;
    String name = "Ashley";
    String surname = "Miller";
    QaTester() {

        System.out.println("QaTester child class constructor çalıştı :) ");
        System.out.println(this.offer);
        System.out.println(super.offer);
        System.out.println(this.cohort);
        System.out.println(super.cohort);
        System.out.println(this.batch);
        // System.out.println(super.batch);
        super.name="Jack";
        System.out.println(this.name);
        System.out.println(super.name);
        this.departman="sales force";
        System.out.println(this.departman);
        System.out.println(super.departman);
        System.out.println(this.surname);
        //System.out.println(super.surname);
    }

    public static void main(String[] args) {
        QaTester qa = new QaTester();
    }
}
  
```

extends

```

class Clarusway {
    int offer = 101;
    int cohort = 1;
    String name = "Hardy";
    String departman = "Java developer";

    Clarusway() {
        System.out.println("Clarusway class consturtor çalıştı :) ");
    }
}
  
```



Inheritance'da Class Member Call

```
public class QaTester extends Clarusway {  
  
    QaTester() {  
        System.out.println("QaTester child class constructor çalıştı :) ");  
        super();  
    }  
}
```

CTE

```
public class QaTester extends Clarusway {  
  
    QaTester() {  
        super();  
        System.out.println("QaTester child class constructor çalıştı :) ");  
    }  
}
```

Compile



Inheritance'da Class Member Call

Output nedir?

```
class Clarusway {  
    public void getOffer() {  
        System.out.println("Clarusway class ");  
    }  
}  
  
public class QaTester extends Clarusway {  
    QaTester() {  
        System.out.println("QaTester class ");  
        super.getOffer();  
    }  
  
    public static void main(String[] args) {  
        QaTester qa = new QaTester();  
        qa.getOffer();  
    }  
}
```

QaTester class
Clarusway class
Clarusway class



Inheritance'da Data Type Kullanımı

```
public class QaTester extends Clarusway {
    String tool;
    int maas;

    public static void main(String[] args) {
    }
}
```

extends

```
public class Clarusway {

    public String path;
    public String isim;
    public String motto;
}
```

extends

```
public class Java extends QaTester {
    String concept;
    int task;

    public static void main(String[] args) {
    }
}
```

Java class'dan parent class data type'dan obj create edilebilir.

Obj create etmek için parent class data type kullanılırsa daha kapsamlı tanımlama yapılır.

Ancak child ve parent class'a ait variable'ler kullanılabilir.

Aynı isimde iki method varsa data type'ına göre belirleyici olur.

```
public class Java extends QaTester {
    String concept;
    int task;

    public static void main(String[] args) {
        Java javaDev = new Java();
        javaDev.concept = "oop"; // Java data type
        javaDev.tool = "appium"; // QaTester data type
        javaDev.isim = "Hardy"; // Clarusway data type

        QaTester tester = new QaTester();
        tester.tool = "selenium"; // QaTester data type
        tester.path = "full stack automation engineer"; // Clarusway data type

        Clarusway kurs = new Clarusway();
        kurs.motto = "way to reinvent yourself"; // Clarusway data type
    }
}
```



Inheritance'da Data Type Kullanımı

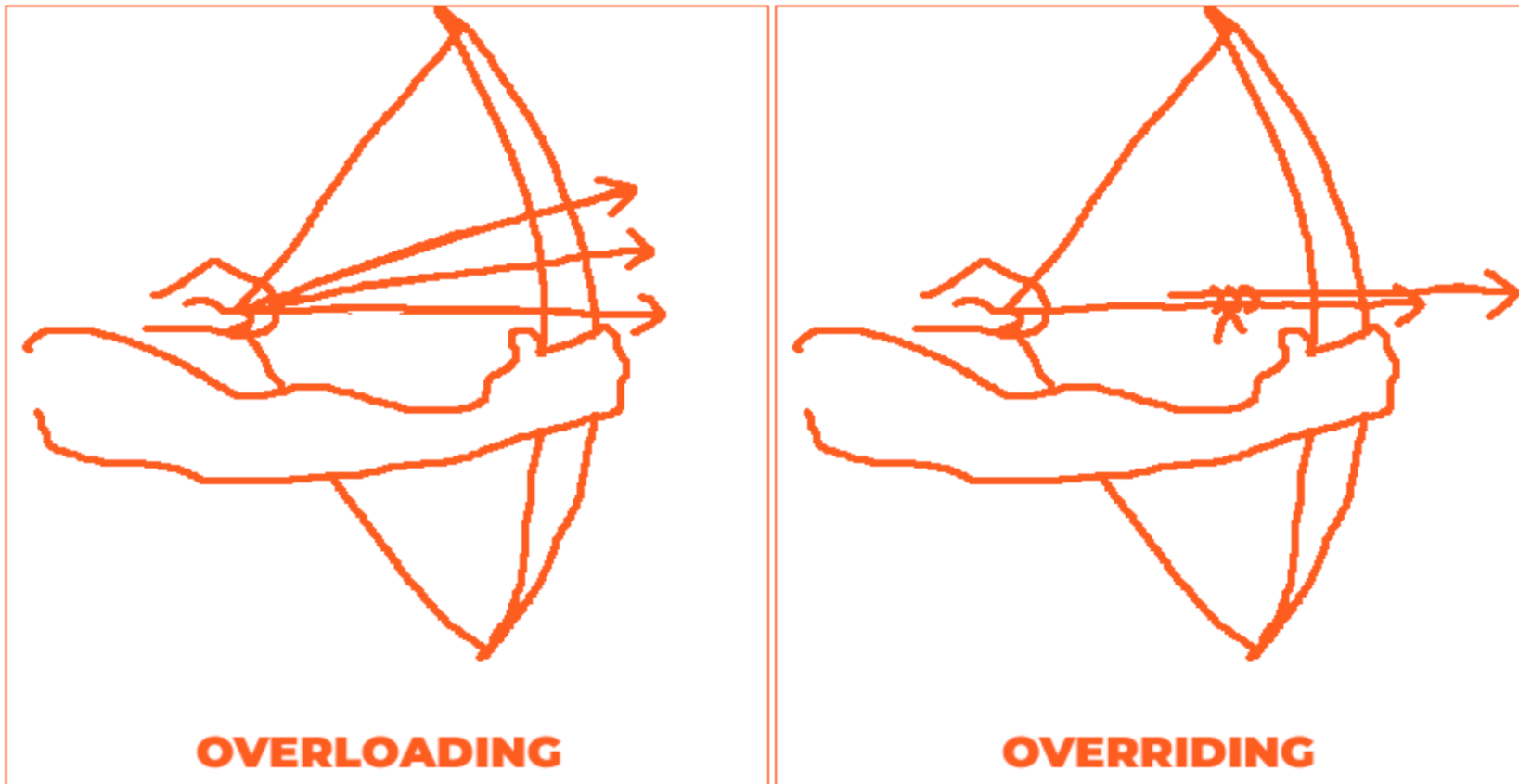
Output nedir?

```
class Clarusway {  
    Clarusway() {  
        System.out.println("Clarusway constructor :) ");  
    }  
  
    public void getOffer() {  
        System.out.println("Clarusway class offer method ");  
    }  
}  
  
class QaTester extends Clarusway {  
    public void getOffer() {  
        System.out.println("QaTester class offer method ");  
    }  
}  
  
class Sdet {  
    public static void main(String[] args) {  
        Clarusway qa = new QaTester();  
        qa.getOffer();  
    }  
}
```

Clarusway constructor :)
QaTester class offer method



Overriding



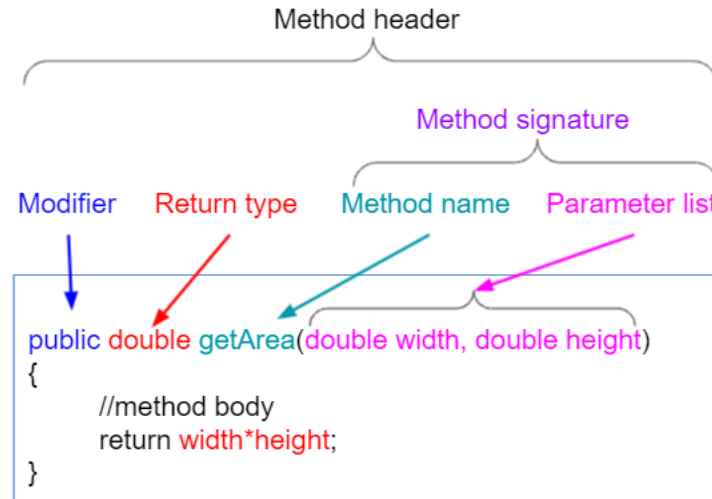
Overriding

Method Signature (Method İmzası)

Method signature; method name ve parametrelerin tipleriyle birlikte sayısını olduğu bir yapıdır. Unutulmaması gereken durum, method signature return type dahil olmamasıdır.

Method Signature Değiştirmek İçin;

- 1- Parametre sayısı değiştirildiğinde **method signature** değişir.
- 2- Farklı data type'daki parametrelerin yerleri değiştirildiğinde **method signature** değişir.



Overloading

Method name aynı farklı methodlar tanımlamak için,

1- Method Signature'ini değiştirilerek. (Overloading)

Overload bir metodun aşırı yüklenmesi anlamına gelir. Örneğin programımızda `selamVer` adında bir metod olsun. Bu metodu `selamVer()` şeklinde yazarak ekrana sadece selam yazdırabiliriz. Aynı şekilde başka bir metod olarak `selamVer(String isim)` şeklinde tanımlayabiliriz. Programımızda `selamVer` metodunu bir string değer ile çağırırsak ikinci metod çağırılır eğer parametresiz çağırırsak birinci metod çağırılır. Java burada metodun aldığı parametrelere bakarak hangi metodun çağırılacağına karar verir. Bu işleme **overload** denir. Burada dikkat edilmesi gereken konular ise parametrelerin sayısı, tipi ve sırası farklı ise kodumuz sorunsuz çalışacaktır. Ancak aynı parametreler ile aynı isimde iki metod yazarsa burada hata alırız. Bir diğer konu ise Java dönüş değerine göre metotları ayırt edemez. Yani `void selamVer()` metodu ve `String selamVer()` metodunu programımıza yazarsa Java bu ikisi arasındaki farkı anlayamaz ve hata verir. Çünkü `selamVer()` metodunu çağırduğumuz yerde hangi dönüş değerini beklediğimizi belirtmek zorunda değilizdir.

* TRICK :)

- Bir class'da aynı method iki defa tanımlanamaz. Aynı Class'da tanımlanan methodların **method signature** değiştirilmezse hata verir.
- Parent ve child class'da aynı method signature sahip iki method tanımlanırsa sadece bir method run edilir.
- **Method Body'nin** değişmesi zorunlu değildir.



Overloading

```
// Overload

public void selamVer() {
    System.out.println("Selam");
}

public void selamVer(String isim) {
    System.out.println("Selam " + isim);
}

public void selamVer(String isim, String soyisim) {
    System.out.println("Selam " + isim + " " + soyisim);
}
```



Overriding

Method name aynı farklı methodlar tanımlamak için,

2- Method Signature aynı fakat method body farklı method'lar tanımlamak. (Overriding)

Override bir metodun tekrardan yazılması(ezmek) anlamına gelir. **inheritance** ile aldığımız bir metodu değiştirmek istersek o zaman override etmemiz yani yeniden yazmamız gerekir. Interface yapısı ile aldığımız metotları da zorunlu olarak tekrardan yazmamız zaten gereklidir. Bu işleme override denir.



Overloading

```
public class Ornek extends Ornek2 {  
  
    // Override  
  
    @Override  
    public void hareketEt() {  
  
        System.out.println("hareketEt metodu override edildi");  
    }  
}  
  
class Ornek2 {  
  
    public void hareketEt() {  
  
        System.out.println("hareket edildi");  
    }  
}
```



Overloading

Java Overriding Kullanımının Avantajları

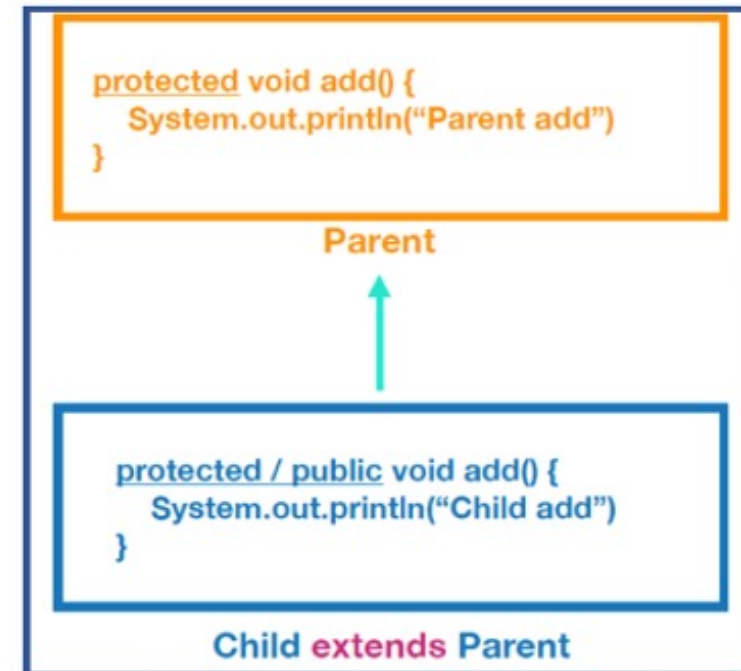
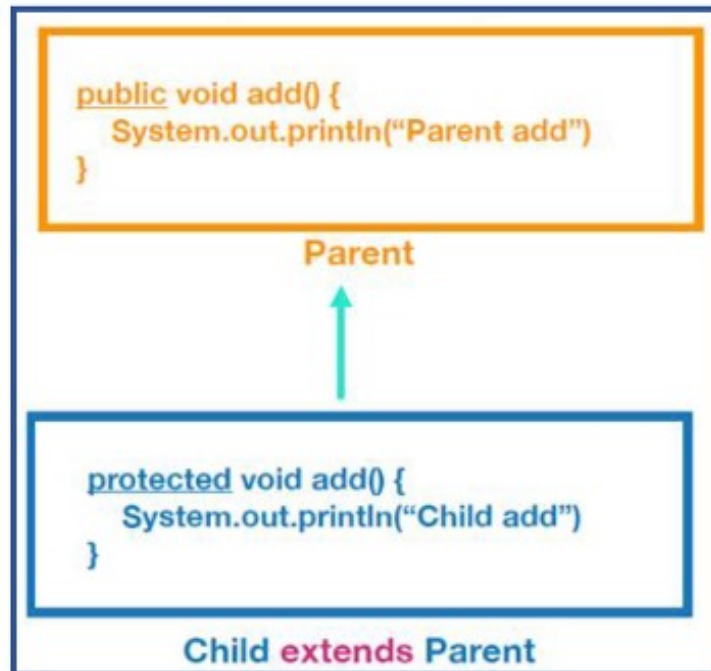
- Temiz, sade ve anlaşılır kod satırları oluşturmamıza imkan sağlar.
- Bir class'ın hangi yapıda duruş sergileyeceğini ve metotların işlevlerinin nasıl uygulanacağını tanımlamamıza imkan sağlar.

Java'da Override (Geçersiz Kılma) Kuralları

- superclass ve subclass'ların aynı
 - ❖ Method name,
 - ❖ Return (covariant) type,
 - ❖ parametre listesine sahip olmalıdır.
- Metotlar, yalnızca alt sınıf (**subclass**) tarafından miras alınmışlarsa override işlemi uygulanabilir.
- Bir metot miras alınmıyorsa override işlemi uygulanamaz.
- Constructor metotlarda override (**geçersiz kılma**) işlemi uygulanamaz.
- Üst sınıf içerisinde yer alan **private** metotlar override işlemine tabi tutulmaz. Aksi takdirde hata alınır.
- Override Metotların acces levellerini yükseltebilir ama düşüremez. (**public** > **protected** > **default** > **private**)
- **Static** ve **Final** olarak tanımlı metotları **override** edilemez.
- Subclass method'u (overriding method), superclass method'un (overridden method) throw edip etmediğine bakmaksızın **Compile Time Exception Throw** edebilir. Ancak parent class'da throw edilen exception'dan daha geniş olamaz.
- abstract olmayan class **abstract class**'a extend ediliyorsa yada **interface** implement ediliyorsa abstract method'ların hepsii override edilmelidir.



Overriding & Access Level



Overriding

Java'da Override Kuralları



Overriding

OVERLOADING VE OVERRIDING KARŞILAŞTIRMASI

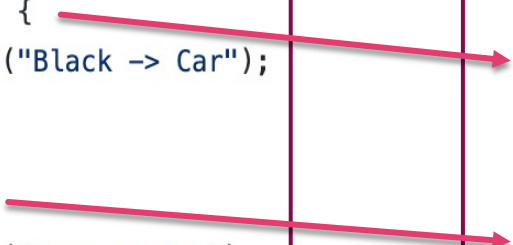
- 1- **Overloading** metotların gövdeleri farklı olabilir. Aynı şekilde **Overriding** işleminde de metotlar farklı gövdelere sahip olabilirler. Böylece hem **Overload** hem de **Override** metotlar farklı işlevleri yerine getiriyor olabilir.
- 2- **Overloading** işleminde metot isimleri birebir aynıdır. Aynı şekilde **Overriding** işleminde de metot isimleri birebir aynıdır.
- 3- **Overloading** işleminde metotların parametre listeleri farklı olmak zorundadır. **Overriding** işleminde ise metotların parametre düzenleri de birebir aynıdır.
- 4- **Overloading** aynı sınıf içerisindeki metotlar arasında görülebilir. **Overriding** ise **SuperClass** ve **SubClass** arasındaki metotlarda görülür. (Not : **Overloading** ayrıca **SuperClass** ve **SubClass** arasındaki metotlarda da görülebilir. Yani her iki sınıfta da aynı isimde ve farklı parametre yapısına sahip metot bulunabilir. Bu maddeyi bu şekilde yazmamızın sebebi Overriding için sadece tek bir sınıfın söz konusu olamayacağını vurgulamak istememiz.)
- 5- **Overloading** işleminde kullanılacak olan metot **compile time'da** belirlenir. **Overriding** işleminde ise kullanılacak metot çalışma zamanında belirlenir.
- 6- **Overloading** hataları **Compile Time'da** ortaya çıktığından dolayı düzeltilmeleri kolaydır. **Overriding** hataları ise **Run Time'da** ortaya çıktığından dolayı daha ciddi problemlere sebebiyet verebilir.
- 7- **Overloading** access modifier ve return type istenilen biçimde kullanılabilir ama **Overriding** bu işlemler sınırlandırılmıştır.



Overriding

```
class Car {  
    public void getColor() {  
        System.out.println("Black -> Car");  
    }  
  
    public void move() {  
        System.out.println("Move -> Car");  
    }  
}
```

```
class Bmw extends Car {  
  
    @Override  
    public void getColor() {  
        System.out.println("Blue -> Bmw");  
    }  
  
    @Override  
    public void move() {  
        System.out.println("Move -> Bmw");  
    }  
}
```



```
public class Main {  
    public static void main(String[] args) {  
        Bmw bmw = new Bmw();  
        bmw.move();  
        bmw.getColor();  
  
        //Output :  
        /*  
        * Move -> Bmw  
        * Blue -> Bmw  
        * */  
    }  
}
```



Overriding

```
class arac {//arac sınıfımız superclass dır.  
    public void goster() {//metodumuzu oluşturduk.  
        System.out.println("Araç motorları");  
    }  
}
```

```
//arac sınıfımızı miras alarak clio sınıfımızı oluşturduk.  
class clio extends arac {  
  
    //override işlemimizi yaparak metodumuzu oluşturduk.  
    @Override  
    public void goster() {  
        System.out.println("Clio = 1.6 motor");  
    }  
}
```

```
//arac sınıfımızı miras alarak corolla sınıfı oluşturduk.  
class corolla extends arac {  
  
    //override işlemimizi yaparak metodumuzu oluşturduk  
    @Override  
    public void goster(){  
        System.out.println("Corolla = 1.2 motor");  
    }  
}
```

```
//Main sınıfımızı oluşturduk.  
class Main {  
    public static void main(String[] args) {  
        //nesnelerimizi oluşturduk  
        clio reno = new clio();  
        corolla toyota = new corolla();  
  
        reno.goster();  
        toyota.goster();  
    }  
}
```

Ekran Çıktısı

```
1 Clio = 1.6 motor  
2 Corolla = 1.2 motor
```



Overriding Kuralları

Java'da **override** işlemini yaparken, parent sınıf (super class) metoduna erişmek için **super** keyword kullanılır.

```
1 class arac {//arac sınıfımız superclass dır.
2     public void goster() {//metodumuzu oluşturduk.
3         System.out.println("Araç motorları");
4     }
5 }
6
7 //arac sınıfımızı miras alarak clio sınıfımızı oluşturduk.
8 class clio extends arac {
9
10    //override işlemimizi yaparak metodumuzu oluşturduk.
11    @Override
12    public void goster() {
13        //super metodumuzu kullandık.
14        super.goster();
15        System.out.println("Clio = 1.6 motor");
16    }
17 }
```



Polymorphism

Polymorphism = Overloading + Overriding

Polymorphism çok biçimlilik demektir. **Polymorphism inheritance** konusu ile iç içe geçmiş bir konudur. **Inheritance** ile oluşturduğumuz class'ların birbiri yerine kullanılabilmesini sağlar. Örneğin Canli adında bir class'ımız olsun. Bitki, Hayvan ve İnsan adındaki class'ları da Canli class'ından extends edelim. **Polymorphism** ile Canli class'ından bir object referansı ile bitki, hayvan ve insan class'ından oluşturduğumuz objeleri gösterebiliriz. örneğin bizim bitki, hayvan ve insan için aynı işi yapmasını istediğimiz bir method olsun. Normal şartlarda bu iş için 3 ayrı metot yazıp kod tekrarı yapmamız gerekecekti ancak **Polymorphism** sayesinde tek bir metot yazıp bu 3 ayrı obje için sadece canli class'ından bir obje alabiliriz. Daha sonra bu metoda insan, hayvan veya bitki sınıfından bir obje gönderildiğinde hangi obje gönderilmiş olursa olsun o obje gibi davranış sergileyecektir.

Superclass objesinin **subclass** objesini göstermesine **upcasting** nedir. Bunun tam tersi de mümkündür. Yani insan class'ından bir obje canli class'ından bir nesneyi gösterebilir. Bu işleme de **downcasting** denir. Ancak bu işlemde dikkat edilmesi gereken yer bu işlemi gerçekleştirirken subclass tipinin belirtilmesi gerektiğidir.

* **TRICK :) Object'lerin gerektiğinde şekilden şekile girip farklı bir object gibi davranabilmesine Polymorphism** denir.



Polymorphism

Java **Polymorphism** Avantajları ;

- Birden çok veri türünü depolamak için tek değişken kullanılabilir.
 - Farklı işlemler arasındaki bağlantıyı azaltır.
 - Kodun tekrar kullanılması sağlanır.
 - Yapılan programda hataları ayıklamak daha kolay olur.
-
- ❖ **Overloading** → **Compile Time (Static)** polymorphism'dir. **Overloading** ile aynı name ve body'e sahip, farklı parametrelerde birden fazla method tanımlanabilir.
 - ❖ Method **Overriding** → **Run Time (Dynamic)** polymorphism'dir. **Overriding** ile aynı name ve parametrelere, farklı body'de birden fazla method tanımlanabilir.



Polymorphism

```
class Kuruyemis {  
    public void goruntu()  
    {  
        System.out.println("Kuruyemişler");  
    }  
}
```

```
//Override ederek class'larımızı oluşturduk ve içlerine metotlarımızı tanımladık.  
class Fındık extends Kuruyemis  
{  
    @Override  
    public void goruntu() {  
        System.out.println("Fındık (Tadı güzelmiş he)");  
    }  
}  
class Ceviz extends Kuruyemis  
{  
    @Override  
    public void goruntu() {  
        System.out.println("Ceviz");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
  
        //kuruyemis class'ından bir object create edildi.  
        Kuruyemis kryms = new Kuruyemis();  
        kryms.goruntu();  
  
        //Fındık class'ından bir object create edildi.  
        Fındık fndk = new Fındık();  
        fndk.goruntu();  
  
        //Ceviz class'ından bir object create edildi.  
        Ceviz cvz = new Ceviz();  
        cvz.goruntu();  
  
    }  
}
```

Ekran Çıktısı

```
1 Kuruyemişler  
2 Fındık (Tadı güzelmiş he)  
3 Ceviz
```



Polymorphism



```
2 public class ItCompany {  
3  
4     public void getDepartmant(String dep) {  
5         System.out.println("ItCompany class " + dep);  
6     }  
7 }  
8  
9 class QaTester extends ItCompany {  
10  
11     public int getDepartmant(String dep) {  
12         System.out.println("QaTester class " + dep);  
13         return 7;  
14     }  
15  
16     public static void main(String[] args) {  
17         QaTester tester = new QaTester();  
18         tester.getDepartmant("UI testing");  
19     }  
20 }
```

Output : CTE



Polymorphism



```
2 public class ItCompany {
3
4     public void getDepartment(String dep) {
5         System.out.println("ItCompany class " + dep);
6     }
7 }
8
9 class QaTester extends ItCompany {
10
11     protected void getDepartment(String dep) {
12         System.out.println("QaTester class " + dep);
13     }
14
15
16     public static void main(String[] args) {
17         QaTester tester = new QaTester();
18         tester.getDepartment("UI testing");
19     }
20 }
```

Output : CTE



Polymorphism



```
2 public class ItCompany {
3
4     public void getDepartment() {
5         System.out.println("ItCompany class ");
6     }
7 }
8
9 class QaTester extends ItCompany {
10
11     public void getDepartment() {
12         System.out.println("QaTester class ");
13     }
14
15     public static void main(String[] args) {
16         QaTester tester = new QaTester();
17         tester.getDepartment();
18     }
19 }
20 }
```

Output : QaTester Class



Polymorphism



```
2 public class ItCompany {
3
4     protected final void getDepartmant() {
5         System.out.println("ItCompany class ");
6     }
7 }
8
9 class QaTester extends ItCompany {
10
11     protected final void getDepartmant() {
12         System.out.println("QaTester class ");
13     }
14
15     public static void main(String[] args) {
16         QaTester tester = new QaTester();
17         tester.getDepartmant();
18     }
19 }
20 }
```

Output : CTE



Polymorphism



```
2  class ItCompany {
3      public void getDepartmant() {
4          System.out.println("ItCompany class ");
5      }
6  }
7
8  class QaTester extends ItCompany {
9      public void getDepartmant() {
10         System.out.println("QaTester class ");
11     }
12 }
13
14
15 class test {
16     public static void main(String[] args) {
17         ItCompany tester = new QaTester();
18         tester.getDepartmant();
19     }
20 }
```

Output : RTE : Main method not found in class QaTester



Polymorphism



```
2  class ItCompany {
3      public Integer getOffer() {
4          System.out.println("ItCompany class ");
5          return 101;
6      }
7  }
8
9  class QaTester extends ItCompany {
10     public Integer getOffer() {
11         System.out.println("QaTester class ");
12         return 99;
13     }
14
15     public static void main(String[] args) {
16         ItCompany it = new ItCompany();
17         QaTester tester = new QaTester();
18         System.out.println(it.getOffer().toString() + ", " + tester.getOffer().toString());
19     }
20 }
```

Output :
ItCompany class
QaTester class
101, 99



Polymorphism



What will be the output of the following program?

```
public class Outer {  
    public static void main(String args[]) {  
        Computer mouse = new Laptop();  
        System.out.println(mouse.getValue(100, 200));  
    }  
}  
class Notebook {  
    int getValue(int a, int b) {  
        if (a > b)  
            return a;  
        else  
            return b;  
    }  
}  
class Computer extends Notebook {  
    int getValue(int a, int b) {  
        return a * b;  
    }  
}  
class Laptop extends Computer {  
    int getValue(int a, int b) {  
        return b - a;  
    }  
}
```



Polymorphism



What will be the output of the following program?

```
public class Product {  
    public static void main(String[] args) {  
        M m = new M();    M n = new N();  
        M o = new O();    O oo = new O();  
        m.product(3);    n.product(3);  
        oo.product(3);  
    }  
}  
class M {  
    int product(int i) {  
        int result = i * i;  
        System.out.print("{ " + i + ", " + result + "}~");  
        return result;  
    }  
}  
class N extends M {  
    int product(int i) {  
        int result = i + i;  
        System.out.print("[ " + i + ", " + result + "]~");  
        return result;  
    }  
}  
class O extends M {  
    int product(int i) {  
        int result = i * 2;  
        System.out.print("(" + i + ", " + result + ")~");  
        return result;  
    }  
}
```



THANKS!

Any questions?

HalUk Bilgin | JAVA Backend Developer

BAŞARI GAYRETE AŞIKTIR 😊

